JAX-RS — JAVA API FOR RESTFUL SERVICES

Hands-On Lab Workbook

Platform: Java 11 LTS | Tomcat 9.0 | Jersey 2.41 | Maven 3.8+
Package namespace: javax.ws.rs.* (NOT jakarta.ws.rs.*)
Starter project: jaxrs-lab-starter.zip (import as Maven project)
Labs: 4 Labs — approx. 4 hours total

© Training Materials — All Rights Reserved

Lab Overview and Prerequisites

Package Namespace — Important

All code in these labs uses the javax.ws.rs.* package namespace.
This is correct for JAX-RS 2.1 / Jersey 2.x running on Java 11 with Tomcat 9.

The jakarta.ws.rs.* namespace is for Jakarta EE 9+ / Tomcat 10 only.
Do NOT use jakarta.* imports in these labs.

Correct imports:

```
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.Response;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.UriInfo;
import javax.ws.rs.ext.ExceptionMapper;
import javax.ws.rs.container.ContainerRequestFilter;
```

Labs in This Workbook

Lab	Title	Key Topics	Section	Time
1	JAX-RS Application Setup	@ApplicationPath, JAX-RS lifecycle, content negotiation	Sec 1–2	40 min
2	CRUD Resource Methods	GET/POST/PUT/DELETE, @PathParam, @QueryParam, Response	Sec 3	55 min
3	Sub-Resources and Locators	Stock/activate sub-resources, ReviewResource locator	Sec 4	45 min
4	Providers and Filters	ExceptionMapper, ContainerRequestFilter, MessageBodyWriter	Sec 5	50 min

Quick Start

1. Unzip jaxrs-lab-starter.zip and import as a Maven project.
2. Run: mvn clean package -DskipTests — build must succeed.
3. Run: mvn test — all smoke tests must PASS.
4. Deploy: cp target/jaxrs-lab.war $CATALINA_HOME/webapps/
5. Test: curl http://localhost:8080/jaxrs-lab/api/health
6. All code must import from javax.ws.rs.*, NOT jakarta.ws.rs.*

© Training Materials — All Rights Reserved

LAB 1

JAX-RS Application Setup and Configuration

Section: Sections 1 & 2

40 min

Objectives

- Verify the ShopApplication and HealthResource are working end-to-end
- Understand what @ApplicationPath sets and how it composes with @Path
- Switch from classpath-scanning to manual class registration with getClasses()
- Understand content negotiation using the Accept header

Package Namespace — Important

All code in these labs uses the javax.ws.rs.* package namespace.
This is correct for JAX-RS 2.1 / Jersey 2.x running on Java 11 with Tomcat 9.

The jakarta.ws.rs.* namespace is for Jakarta EE 9+ / Tomcat 10 only.
Do NOT use jakarta.* imports in these labs.

Correct imports:

```
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.Response;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.UriInfo;
import javax.ws.rs.ext.ExceptionMapper;
import javax.ws.rs.container.ContainerRequestFilter;
```

Task 1.1 — Build and Deploy

```
mvn clean package -DskipTests
cp target/jaxrs-lab.war $CATALINA_HOME/webapps/
$CATALINA_HOME/bin/startup.sh   # or startup.bat on Windows

# Confirm deployment — look for this line in catalina.out:
# INFO: Deployment of web application archive [jaxrs-lab.war] has finished

curl http://localhost:8080/jaxrs-lab/api/health
# Expected:
{"status":"UP","service":"ShopAPI","version":"1.0","timestamp":"..."}
```

Paste the health check JSON response here:

Task 1.2 — Understand the URI Structure

The full URI for any resource is composed from three parts. Fill in the table for the HealthResource:

© Training Materials — All Rights Reserved

Part	Value	Where declared
Tomcat context root	/jaxrs-lab	WAR filename
Application base path	??????	@ApplicationPath on ShopApplication
Resource path	/health	@Path on HealthResource
Full URI	http://localhost:8080/?????	Composed

Task 1.3 — Add a Second Resource Class

Create a new class ApiInfoResource that responds to GET /api/info with a JSON map containing the API name, version, and a list of available endpoints. Register it alongside HealthResource in the Tomcat deployment (classpath scanning picks it up automatically):

```
package com.example.api.resource;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

@Path("/info")
public class ApiInfoResource {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public Map<String, Object> info() {
        Map<String, Object> info = new LinkedHashMap<>();
        info.put("api",       "ShopAPI");
        info.put("version",   "1.0");
        info.put("endpoints", List.of("/health","/info","/products"));
        return info;
    }
}

curl http://localhost:8080/jaxrs-lab/api/info
```

Task 1.4 — Switch to Manual Registration

Open ShopApplication.java and uncomment the getClasses() method. Add HealthResource and ApiInfoResource to the set. Redeploy and verify both endpoints still work:

```
@ApplicationPath("/api")
public class ShopApplication extends Application {

    @Override
    public java.util.Set<Class<?>> getClasses() {
        return java.util.Set.of(
            com.example.api.resource.HealthResource.class,
            com.example.api.resource.ApiInfoResource.class
            // Add ProductResource and ReviewResource here in later labs
        );
    }
}
```

© Training Materials — All Rights Reserved

```
// After redeployment — both must still respond:
curl http://localhost:8080/jaxrs-lab/api/health
curl http://localhost:8080/jaxrs-lab/api/info
```

Q1

What happens to ProductResource once you switch to manual getClasses() registration? Why does this happen? What would you need to do to make /api/products work again?

Your answer:

Task 1.5 — Content Negotiation

The HealthResource only produces APPLICATION_JSON. Add a second @GET method that produces TEXT_PLAIN, and use the Accept header to request each variant:

```
// Add to HealthResource:
@GET
@Produces(MediaType.TEXT_PLAIN)
public String healthText() {
    return "UP";
}

// Test both variants:
curl -H 'Accept: application/json' http://localhost:8080/jaxrs-lab/api/health
# → JSON object

curl -H 'Accept: text/plain' http://localhost:8080/jaxrs-lab/api/health
# → UP

curl http://localhost:8080/jaxrs-lab/api/health
# → which does Jersey choose when Accept is not set?
```

Which variant does Jersey return when Accept is */* (not specified)? Why?

Lab 1 Checkpoint

- ✓ GET /api/health returns {"status":"UP",...}
- ✓ GET /api/info returns API name, version, and endpoint list
- ✓ Manual getClasses() works — only registered resources respond
- ✓ Content negotiation selects JSON vs plain text via Accept header

© Training Materials — All Rights Reserved

LAB 2

CRUD Resource Methods — ProductResource

Section: Section 3

55 min

Objectives

- Implement all five CRUD operations in ProductResource using javax.ws.rs.*
- Use @QueryParam, @PathParam, @DefaultValue, and @Context UriInfo correctly
- Build Response objects with the correct status codes and Location headers
- Test each endpoint with curl

Package Namespace — Important

All code in these labs uses the javax.ws.rs.* package namespace.
This is correct for JAX-RS 2.1 / Jersey 2.x running on Java 11 with Tomcat 9.

The jakarta.ws.rs.* namespace is for Jakarta EE 9+ / Tomcat 10 only.
Do NOT use jakarta.* imports in these labs.

Correct imports:

```
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.Response;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.UriInfo;
import javax.ws.rs.ext.ExceptionMapper;
import javax.ws.rs.container.ContainerRequestFilter;
```

Task 2.1 — GET /api/products (list with filtering)

Open ProductResource.java. Implement the listAll() method — the parameters are already declared, just call the repository and return 200:

```
// All imports already in place: import javax.ws.rs.*;

@GET
public Response listAll(
        @QueryParam("category")                String  category,
        @QueryParam("page")  @DefaultValue("0") int     page,
        @QueryParam("size")  @DefaultValue("20") int     size,
        @QueryParam("active") @DefaultValue("true") boolean activeOnly) {

    List<Product> products = repo.find(category, page, size, activeOnly);
    return Response.ok(products).build();
    // Note: empty list → 200 OK with [], NOT 404
}

# Test:
curl http://localhost:8080/jaxrs-lab/api/products
curl 'http://localhost:8080/jaxrs-lab/api/products?category=books'
curl 'http://localhost:8080/jaxrs-
lab/api/products?category=books&active=false'
curl 'http://localhost:8080/jaxrs-lab/api/products?page=0&size=2'
```

© Training Materials — All Rights Reserved

Paste the JSON response for ?category=books here:

Task 2.2 — GET /api/products/{id}

```
@GET
@Path("{id}")
public Response getById(@PathParam("id") String id) {
    Product product = repo.findById(id);
    if (product == null) {
        return Response.status(Response.Status.NOT_FOUND)
            .entity(Map.of("error", "Product not found: " + id,
                           "status", 404))
            .build();
    }
    return Response.ok(product).build();
}

# Test:
curl http://localhost:8080/jaxrs-lab/api/products/P001
curl -v http://localhost:8080/jaxrs-lab/api/products/DOES_NOT_EXIST
# Expected: 404 with JSON error body
```

Task 2.3 — POST /api/products

```
@POST
public Response create(Product product, @Context UriInfo uriInfo) {
    // Basic validation
    if (product.getName() == null || product.getName().isBlank()) {
        return Response.status(Response.Status.BAD_REQUEST)
            .entity(Map.of("error", "Product name is required",
                           "status", 400))
            .build();
    }
    Product saved = repo.save(product);
    URI location = uriInfo.getAbsolutePathBuilder()
        .path(saved.getId())
        .build();
    return Response.created(location).entity(saved).build();
}

# Test:
curl -v -X POST http://localhost:8080/jaxrs-lab/api/products \
     -H 'Content-Type: application/json' \
     -d '{"name":"The Pragmatic
Programmer","category":"books","price":44.99,"stockLevel":25,"active":true}'
# Expected: 201 Created + Location: http://.../api/products/P007

curl -X POST http://localhost:8080/jaxrs-lab/api/products \
     -H 'Content-Type: application/json' -d '{}'
# Expected: 400 Bad Request
```

Task 2.4 — PUT and DELETE /api/products/{id}

© Training Materials — All Rights Reserved

```
// PUT — full replacement
@PUT
@Path("{id}")
public Response update(@PathParam("id") String id, Product product) {
    Product updated = repo.update(id, product);
    if (updated == null) {
        return Response.status(Response.Status.NOT_FOUND)
            .entity(Map.of("error","Not found","status",404)).build();
    }
    return Response.ok(updated).build();
}

// DELETE
@DELETE
@Path("{id}")
public Response delete(@PathParam("id") String id) {
    if (!repo.delete(id)) {
        return Response.status(Response.Status.NOT_FOUND)
            .entity(Map.of("error","Not found","status",404)).build();
    }
    return Response.noContent().build();   // 204
}

# Test:
curl -v -X PUT http://localhost:8080/jaxrs-lab/api/products/P001 \
     -H 'Content-Type: application/json' \
     -d '{"name":"Clean Code
(Updated)","category":"books","price":42.99,"stockLevel":60,"active":true}'

curl -v -X DELETE http://localhost:8080/jaxrs-lab/api/products/P001
# Expected: 204 No Content

curl -v -X DELETE http://localhost:8080/jaxrs-lab/api/products/MISSING
# Expected: 404
```

Task 2.5 — Content Negotiation on GET /api/products/{id}/summary

Implement getSummary() to return either JSON or plain text depending on the Accept header. Use @Context Request and request.selectVariant() to perform formal content negotiation:

```
import javax.ws.rs.core.Request;
import javax.ws.rs.core.Variant;
import java.util.List;

@GET
@Path("{id}/summary")
@Produces({MediaType.APPLICATION_JSON, MediaType.TEXT_PLAIN})
public Response getSummary(@PathParam("id") String id,
                           @Context Request request) {
    Product p = repo.findById(id);
    if (p == null) return Response.status(404).build();

    List<Variant> variants = Variant.mediaTypes(
        MediaType.APPLICATION_JSON_TYPE,
        MediaType.TEXT_PLAIN_TYPE).build();
    Variant chosen = request.selectVariant(variants);

    if (MediaType.TEXT_PLAIN_TYPE.equals(chosen.getMediaType())) {
        String text = String.format("%s | %s | £%.2f | stock: %d",
            p.getId(), p.getName(), p.getPrice(), p.getStockLevel());
        return Response.ok(text, MediaType.TEXT_PLAIN).build();
    }
```

© Training Materials — All Rights Reserved

```
    // Default: JSON
    return Response.ok(Map.of(
        "id", p.getId(), "name", p.getName(),
        "price", p.getPrice(), "stockLevel", p.getStockLevel()
    )).build();
}

curl -H 'Accept: text/plain' \
     http://localhost:8080/jaxrs-lab/api/products/P001/summary
# → P001 | Clean Code | £39.99 | stock: 50

curl -H 'Accept: application/json' \
     http://localhost:8080/jaxrs-lab/api/products/P001/summary
# → {"id":"P001","name":"Clean Code",...}
```

Q1

What HTTP status code does Jersey return if the Accept header requests a media type that no @Produces method supports (e.g. Accept: text/html on this endpoint)?

Your answer:

Lab 2 Checkpoint

- ✓ GET /api/products returns 200 with all 6 seed products
- ✓ GET /api/products?category=books returns only books
- ✓ GET /api/products/P001 returns 200; /MISSING returns 404
- ✓ POST /api/products returns 201 + Location header
- ✓ PUT /api/products/P001 returns 200 with updated product
- ✓ DELETE /api/products/P001 returns 204; second call returns 404
- ✓ GET /api/products/P001/summary with text/plain returns pipe-delimited string

© Training Materials — All Rights Reserved

LAB 3

Sub-Resources and Sub-Resource Locators

Section: Section 4

45 min

Objectives

- Implement stock-management sub-resource methods on ProductResource
- Implement activate/deactivate action sub-resources
- Implement a sub-resource locator that returns a ReviewResource instance
- Understand the difference between a sub-resource method and a sub-resource locator

Task 3.1 — Stock Sub-Resource Methods

Add GET and PUT stock methods to ProductResource. These are sub-resource methods — they have both @GET/@PUT and @Path on the same method:

```
// GET /api/products/{id}/stock
@GET
@Path("{id}/stock")
@Produces(MediaType.APPLICATION_JSON)
public Response getStock(@PathParam("id") String id) {
    Product p = repo.findById(id);
    if (p == null) return Response.status(404).build();
    return Response.ok(
        Map.of("productId", id, "stockLevel", p.getStockLevel())
    ).build();
}

// PUT /api/products/{id}/stock  body: {"quantity": 50}
@PUT
@Path("{id}/stock")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response updateStock(@PathParam("id") String id,
                            Map<String,Integer> body) {
    Product p = repo.findById(id);
    if (p == null) return Response.status(404).build();
    Integer qty = body.get("quantity");
    if (qty == null || qty < 0) {
        return Response.status(400)
            .entity(Map.of("error","quantity must be >= 0")).build();
    }
    p.setStockLevel(qty);
    repo.update(id, p);
    return Response.ok(
        Map.of("productId", id, "stockLevel", qty)
    ).build();
}

# Test:
curl http://localhost:8080/jaxrs-lab/api/products/P001/stock

curl -X PUT http://localhost:8080/jaxrs-lab/api/products/P001/stock \
     -H 'Content-Type: application/json' -d '{"quantity":99}'
curl http://localhost:8080/jaxrs-lab/api/products/P001/stock
# Expected: stockLevel: 99
```

© Training Materials — All Rights Reserved

Task 3.2 — Activate / Deactivate Sub-Resources

```
// PUT /api/products/{id}/activate
@PUT
@Path("{id}/activate")
@Produces(MediaType.APPLICATION_JSON)
public Response activate(@PathParam("id") String id) {
    Product p = repo.findById(id);
    if (p == null) return Response.status(404).build();
    p.setActive(true);
    repo.update(id, p);
    return Response.ok(
        Map.of("productId", id, "active", true)).build();
}

// PUT /api/products/{id}/deactivate
@PUT
@Path("{id}/deactivate")
@Produces(MediaType.APPLICATION_JSON)
public Response deactivate(@PathParam("id") String id) {
    Product p = repo.findById(id);
    if (p == null) return Response.status(404).build();
    p.setActive(false);
    repo.update(id, p);
    return Response.ok(
        Map.of("productId", id, "active", false)).build();
}

# Test:
curl -X PUT http://localhost:8080/jaxrs-lab/api/products/P006/activate
curl 'http://localhost:8080/jaxrs-lab/api/products?active=false'
# P006 (Old Java EE 5 Book) should no longer appear in active=false results
```

Task 3.3 — Sub-Resource Locator for Reviews

A sub-resource LOCATOR is a method with @Path but NO HTTP verb annotation. Jersey calls it to get the object that handles the rest of the request. Add it to ProductResource and complete ReviewResource:

```
// In ProductResource — the LOCATOR (no @GET/@POST):
@Path("{id}/reviews")
public ReviewResource getReviewResource(@PathParam("id") String id) {
    // Validate the product exists before delegating
    if (!repo.exists(id)) {
        throw new javax.ws.rs.NotFoundException(
            "Product not found: " + id);
    }
    // Jersey will call @GET/@POST methods on the returned object
    return new ReviewResource(id);
}

// In ReviewResource — complete getById() and create():
@GET
@Path("{rid}")
public Response getById(@PathParam("rid") String rid) {
    Review review = reviewRepo.findById(rid);
    if (review == null || !productId.equals(review.getProductId()))
        return Response.status(404).build();
    return Response.ok(review).build();
}

@POST
public Response create(Review review, @Context UriInfo uriInfo) {
```

© Training Materials — All Rights Reserved

```
    review.setProductId(productId);
    Review saved = reviewRepo.save(review);
    URI location = uriInfo.getAbsolutePathBuilder()
        .path(saved.getId()).build();
    return Response.created(location).entity(saved).build();
}

# Test:
curl http://localhost:8080/jaxrs-lab/api/products/P001/reviews

curl -X POST http://localhost:8080/jaxrs-lab/api/products/P001/reviews \
     -H 'Content-Type: application/json' \
     -d '{"rating":5,"comment":"Excellent!","reviewerName":"Dave"}'

curl http://localhost:8080/jaxrs-lab/api/products/MISSING/reviews
# Expected: 404
```

Q1

What is the key difference between a sub-resource METHOD and a sub-resource LOCATOR? When would you choose a locator over direct methods on the parent resource?

Your answer:

Lab 3 Checkpoint

- ✓ GET /api/products/P001/stock — returns {productId, stockLevel}
- ✓ PUT /api/products/P001/stock — updates stock level
- ✓ PUT /api/products/P006/activate — sets P006 active=true
- ✓ GET /api/products/P001/reviews — returns seeded reviews
- ✓ POST /api/products/P001/reviews — adds a review and returns 201
- ✓ GET /api/products/MISSING/reviews — returns 404

© Training Materials — All Rights Reserved

LAB 4

Providers — Exception Mappers, Filters, and MessageBodyWriter

Section: Section 5

50 min

Objectives

- Implement ExceptionMapper<NotFoundException> and ExceptionMapper<Exception>
- Implement a ContainerResponseFilter to add CORS headers
- Implement a @NameBinding ContainerRequestFilter for Bearer token auth
- Implement a MessageBodyWriter<List<Product>> to produce text/csv

Package Namespace — Important

All code in these labs uses the javax.ws.rs.* package namespace.
This is correct for JAX-RS 2.1 / Jersey 2.x running on Java 11 with Tomcat 9.

The jakarta.ws.rs.* namespace is for Jakarta EE 9+ / Tomcat 10 only.
Do NOT use jakarta.* imports in these labs.

Correct imports:

```
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.Response;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.UriInfo;
import javax.ws.rs.ext.ExceptionMapper;
import javax.ws.rs.container.ContainerRequestFilter;
```

Task 4.1 — Exception Mappers

Open provider/Providers.java. Implement NotFoundExceptionMapper and GlobalExceptionMapper:

```
import javax.ws.rs.NotFoundException;
import javax.ws.rs.core.Response;
import javax.ws.rs.ext.ExceptionMapper;
import javax.ws.rs.ext.Provider;

@Provider
class NotFoundExceptionMapper
        implements ExceptionMapper<NotFoundException> {

    @Override
    public Response toResponse(NotFoundException ex) {
        return Response
            .status(Response.Status.NOT_FOUND)
            .entity(Map.of(
                "status", 404,
                "error",  "Not Found",
                "detail", ex.getMessage() != null
                            ? ex.getMessage() : "Resource not found"))
            .type(MediaType.APPLICATION_JSON)
            .build();
```

© Training Materials — All Rights Reserved

```
    }
}

@Provider
class GlobalExceptionMapper implements ExceptionMapper<Exception> {

    @Override
    public Response toResponse(Exception ex) {
        // In production: log ex properly
        System.err.println("[ERROR] Unhandled: " + ex.getMessage());
        return Response
            .status(Response.Status.INTERNAL_SERVER_ERROR)
            .entity(Map.of(
                "status", 500,
                "error",  "Internal Server Error"))
            .type(MediaType.APPLICATION_JSON)
            .build();
    }
}

# Test (before mapper): 404 returns HTML error page from Tomcat
# Test (after mapper):
curl -v http://localhost:8080/jaxrs-lab/api/products/MISSING
# Expected: 404 with JSON body {"status":404,"error":"Not Found",...}
```

Task 4.2 — CORS Response Filter

```
import javax.annotation.Priority;
import javax.ws.rs.Priorities;
import javax.ws.rs.container.ContainerRequestContext;
import javax.ws.rs.container.ContainerResponseContext;
import javax.ws.rs.container.ContainerResponseFilter;
import javax.ws.rs.ext.Provider;

@Provider
@Priority(Priorities.HEADER_DECORATOR)
class CorsFilter implements ContainerResponseFilter {

    @Override
    public void filter(ContainerRequestContext  req,
                       ContainerResponseContext resp) {
        resp.getHeaders().add(
            "Access-Control-Allow-Origin",  "*");
        resp.getHeaders().add(
            "Access-Control-Allow-Methods",
            "GET, POST, PUT, DELETE, OPTIONS");
        resp.getHeaders().add(
            "Access-Control-Allow-Headers",
            "Authorization, Content-Type");
        resp.getHeaders().add(
            "Access-Control-Max-Age",       "86400");
    }
}

# Test — CORS headers should appear on every response:
curl -v http://localhost:8080/jaxrs-lab/api/health
# Expected headers: Access-Control-Allow-Origin: *  etc.
```

Task 4.3 — Bearer Token Authentication Filter (@NameBinding)

© Training Materials — All Rights Reserved

Implement BearerTokenFilter and apply @Secured to the DELETE /api/products/{id} method:

```
import javax.ws.rs.container.ContainerRequestContext;
import javax.ws.rs.container.ContainerRequestFilter;
import javax.ws.rs.core.HttpHeaders;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

@Provider
@Secured                              // only fires on @Secured endpoints
@Priority(Priorities.AUTHENTICATION)
class BearerTokenFilter implements ContainerRequestFilter {

    private static final String VALID = "secret-token-123";

    @Override
    public void filter(ContainerRequestContext ctx) {
        String auth = ctx.getHeaderString(HttpHeaders.AUTHORIZATION);
        if (auth == null || !auth.equals("Bearer " + VALID)) {
            ctx.abortWith(Response
                .status(Response.Status.UNAUTHORIZED)
                .entity(Map.of(
                    "error", "Unauthorized",
                    "status", 401))
                .type(MediaType.APPLICATION_JSON)
                .build());
        }
    }
}

// In ProductResource — annotate the DELETE method:
// import com.example.api.provider.Secured;
@DELETE
@Path("{id}")
@Secured                              // add this annotation
public Response delete(@PathParam("id") String id) { ... }

# Test — missing token:
curl -v -X DELETE http://localhost:8080/jaxrs-lab/api/products/P001
# Expected: 401 Unauthorized

# Test — valid token:
curl -v -X DELETE http://localhost:8080/jaxrs-lab/api/products/P001 \
     -H 'Authorization: Bearer secret-token-123'
# Expected: 204 No Content
```

Task 4.4 — CSV MessageBodyWriter

Implement ProductCsvWriter and add @Produces("text/csv") to ProductResource.listAll():

```
import javax.ws.rs.Produces;
import javax.ws.rs.WebApplicationException;
import javax.ws.rs.core.*;
import javax.ws.rs.ext.MessageBodyWriter;
import javax.ws.rs.ext.Provider;

@Provider
@Produces("text/csv")
class ProductCsvWriter implements MessageBodyWriter<List<Product>> {

    @Override
    public boolean isWriteable(Class<?> type, Type genericType,
```

© Training Materials — All Rights Reserved

```
                                Annotation[] annotations,
                                MediaType mediaType) {
        if (!List.class.isAssignableFrom(type)) return false;
        if (genericType instanceof ParameterizedType pt) {
            return Product.class.equals(
                pt.getActualTypeArguments()[0]);
        }
        return false;
    }

    @Override
    public void writeTo(List<Product> products,
                        Class<?> type, Type genericType,
                        Annotation[] annotations, MediaType mediaType,
                        MultivaluedMap<String,Object> headers,
                        OutputStream entityStream) throws IOException {
        PrintWriter pw = new PrintWriter(
            new OutputStreamWriter(entityStream,
                StandardCharsets.UTF_8));
        pw.println("id,name,category,price,stockLevel,active");
        for (Product p : products) {
            pw.printf("%s,%s,%s,%.2f,%d,%b%n",
                p.getId(), p.getName(), p.getCategory(),
                p.getPrice(), p.getStockLevel(), p.isActive());
        }
        pw.flush();
    }

    @Override
    public long getSize(List<Product> p, Class<?> t, Type g,
                        Annotation[] a, MediaType m) { return -1; }
}

// Also add to ProductResource.listAll():
@GET
@Produces({MediaType.APPLICATION_JSON, "text/csv"})
public Response listAll(...) { ... }

# Test:
curl -H 'Accept: text/csv' \
     http://localhost:8080/jaxrs-lab/api/products
# Expected CSV output:
# id,name,category,price,stockLevel,active
# P001,Clean Code,books,39.99,50,true
# ...
```

Q1

The @NameBinding @Secured annotation is declared in your provider package. Where must you import it when applying it to a resource method in ProductResource? Can it be in a different package?

Your answer:

Q2

© Training Materials — All Rights Reserved

ExceptionMapper<Exception> is registered alongside ExceptionMapper<NotFoundException>. A NotFoundException is thrown. Which mapper handles it? Why?

Your answer:

Lab 4 Checkpoint

- ✓ GET /api/products/MISSING returns 404 JSON (not HTML Tomcat error page)
- ✓ Every response includes Access-Control-Allow-Origin: * header
- ✓ DELETE /api/products/P001 (no token) returns 401
- ✓ DELETE /api/products/P001 with Bearer secret-token-123 returns 204
- ✓ GET /api/products Accept: text/csv returns CSV with headers

© Training Materials — All Rights Reserved

Model Answers — Key Code

Reference implementations. Consult only after attempting each lab.

Lab 2 — listAll()

```
@GET
@Produces({MediaType.APPLICATION_JSON, "text/csv"})
public Response listAll(
        @QueryParam("category")                String  category,
        @QueryParam("page")  @DefaultValue("0") int     page,
        @QueryParam("size")  @DefaultValue("20") int     size,
        @QueryParam("active") @DefaultValue("true") boolean activeOnly) {
    return Response.ok(repo.find(category, page, size, activeOnly)).build();
}
```

Lab 3 — Sub-resource locator

```
@Path("{id}/reviews")
public ReviewResource getReviewResource(@PathParam("id") String id) {
    if (!repo.exists(id))
        throw new javax.ws.rs.NotFoundException("Product not found: " + id);
    return new ReviewResource(id);
}
```

Lab 4 — BearerTokenFilter

```
@Override
public void filter(ContainerRequestContext ctx) {
    String auth = ctx.getHeaderString(HttpHeaders.AUTHORIZATION);
    if (auth == null || !auth.equals("Bearer " + VALID)) {
        ctx.abortWith(Response.status(401)
            .entity(Map.of("error","Unauthorized","status",401))
            .type(MediaType.APPLICATION_JSON).build());
    }
}
```

© Training Materials — All Rights Reserved